

Operációs rendszerek II.

Állományok kezelése



Várkonyiné Kóczy Annamária

Professzor

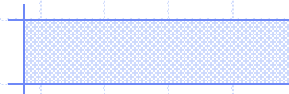
Informatika Tanszék

koczya@uj.ssk

varkonyi-koczy@uni-obuda.hu

Felhasznált irodalom:

- Kóczy-Kondorosi (szerk.): Operációs rendszerek mérnöki megközelítésben
- Tanenbaum: Modern Operating Systems 2nd. Ed.
- Silberschatz, Galvin, Gagne: Operating System Concepts



9. Állományok kezelése

- Alapfogalmak
- Szabad blokkok nyilvántartása
- A lemez blokkjainak allokációja
- Állományok szerkezete
- Könyvtárak szerkezete
- Műveletek
- Osztott állománykezelés
- Hozzáférés szabályozása

Bevezetés

- Minden szg. / rendszernek tudnia kell tárolni és előhívni információt
- Futó program → véges információ mennyiség a munkaterületén (a főmemóriában)
- Kikapcsoláskor elvész a tartalom
- Rendszeres igény, hogy több folyamat is hozzáférjen az információ egy részéhez párhuzamosan



Követelmények a *hosszú távú* információtároláshoz:

- Nagyon nagy mennyiségű adatot legyen képes tárolni
- Az információ túl kell élje az őt használó/létrehozó folyamatot
- Több folyamat egyszerre hozzáférhessen (osztott használat)

Bevezetés (folyt.)

Megoldás:

- Háttértárakon tároljuk hosszú távon
 - File-ban:
 - absztrakciós mechanizmus,
 - állandó, független a folyamat születésétől és halálától
- ↓
- a file-okat az OS kezeli (rendszerhívás!)

Állomány (file)

- A létrehozó által összetartozónak ítélt információk gyűjteménye.
- Több százezer állomány egyidejűleg, egyedi azonosító (név) különbözteti meg őket.
- Az állomány elrejtí a tárolásának, kezelésének fizikai részleteit:
 - Melyik fizikai eszközön található (logikai eszköznevek használata);
 - a perifériás illesztő tulajdonságai;
 - az állományhoz tartozó információk elhelyezkedése a lemezen;
 - az állományban lévő információk hogyan helyezkednek el a fizikai egységen (szektor, blokk);
 - az információ átvitelénél alkalmazott blokkosítást, puffereelést.

Könyvtár

- Az állományok csoportosítása, OS és a felhasználó szerint.
- A könyvtár tartalmát katalógus írja le.

Az állománykezelő feladatai

- információátvitel (állomány és a folyamatok között)
- műveletek (állományokon, könyvtárakon)
- osztott állománykezelés
- hozzáférés szabályozása
 - más felhasználók által végezhető műveletek korlátozása (access control)
 - tárolt információk védelme az illetéktelen olvasások ellen (rejtjelezés, encryption)
 - információ védelme a sérülések ellen, mentés

Az állományrendszer réteges implementációja

Egymásra épülő programrétegek

1. a perifériát közvetlenül kezelő periféria meghajtó
Feladata a tár és a periféria közötti átvitel megvalósítása. (device driver, átvitelt kezdeményező, megszakítást kezelő eljárások)
2. elemi átviteli műveletek rétege
A lineáris címzésű blokkok átvitele, átmeneti tárolása.
3. állományszervezés rétege
Háttértár szabad blokkjainak, illetve az állományhoz tartozó blokkoknak szervezése.
4. logikai állományszervezés
Kezeli a nyilvántartások szerkezetét, azonosító alapján megtalálja az állományt, szabályozza az állományszintű átvitelt.

Állományok tárolása a lemezen

- Lemezterületet blokkonként (néhány szektor) kezeli. Mérete a lapmérethez hasonló megfontolások alapján.
- Az állomány tárolásához blokkok kellenek, ezeket nyomon kell követni.
 - Szabad blokkok nyilvántartása
 - Blokkok allokálása

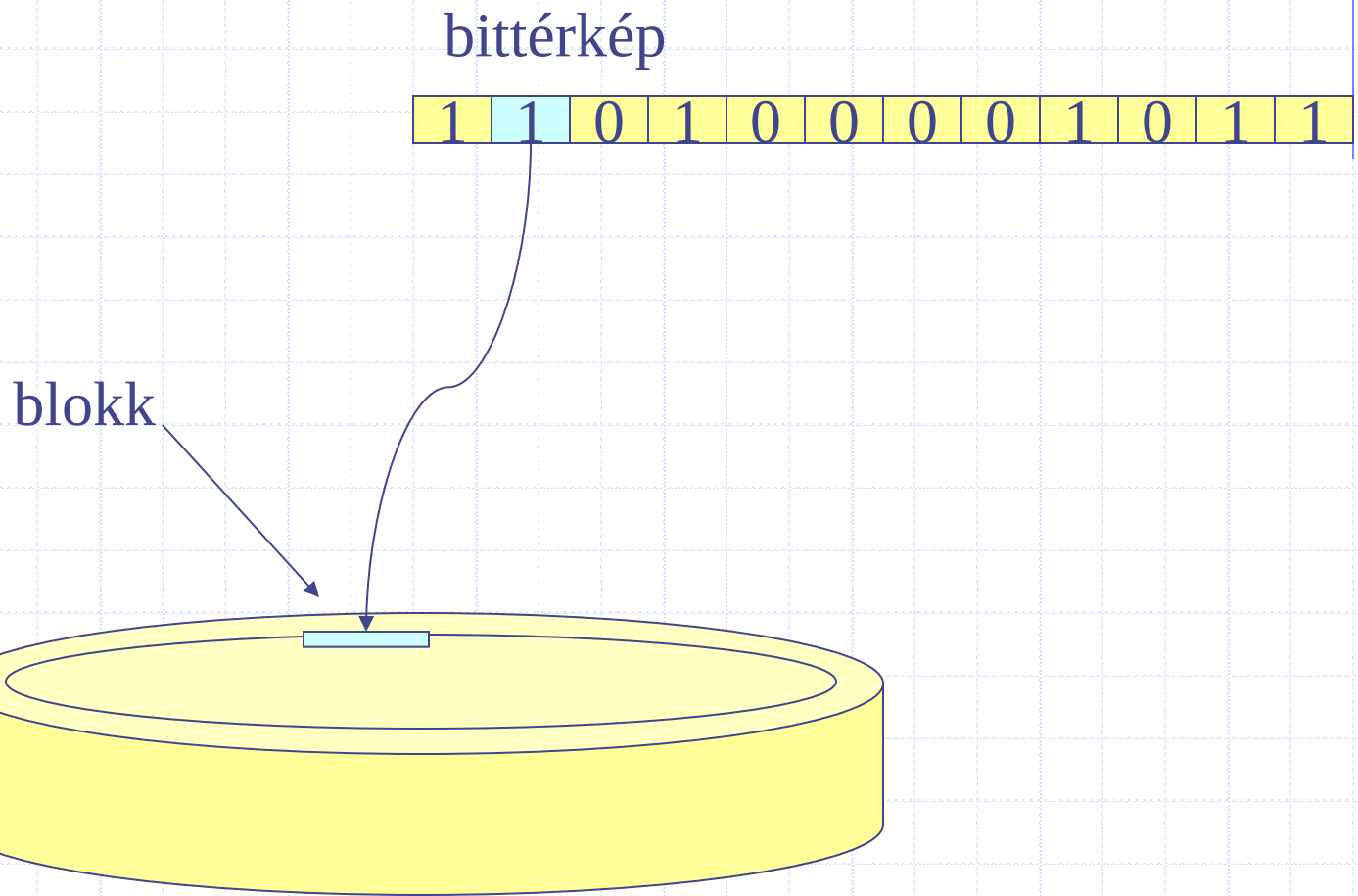
Szabad blokkok nyilvántartása

- Bittérkép
- Láncolt lista
- Szabad helyek csoportjainak listája
- Egybefüggő szabad terület tárolása

Bittérkép

- Minden blokkra egy biten jelzi, hogy szabad-e
- A bittérkép a lemez kijelölt helyén van
- A bitvektort a memóriában kell tárolni. Sokk blokk van, így sok memóriát igényel.

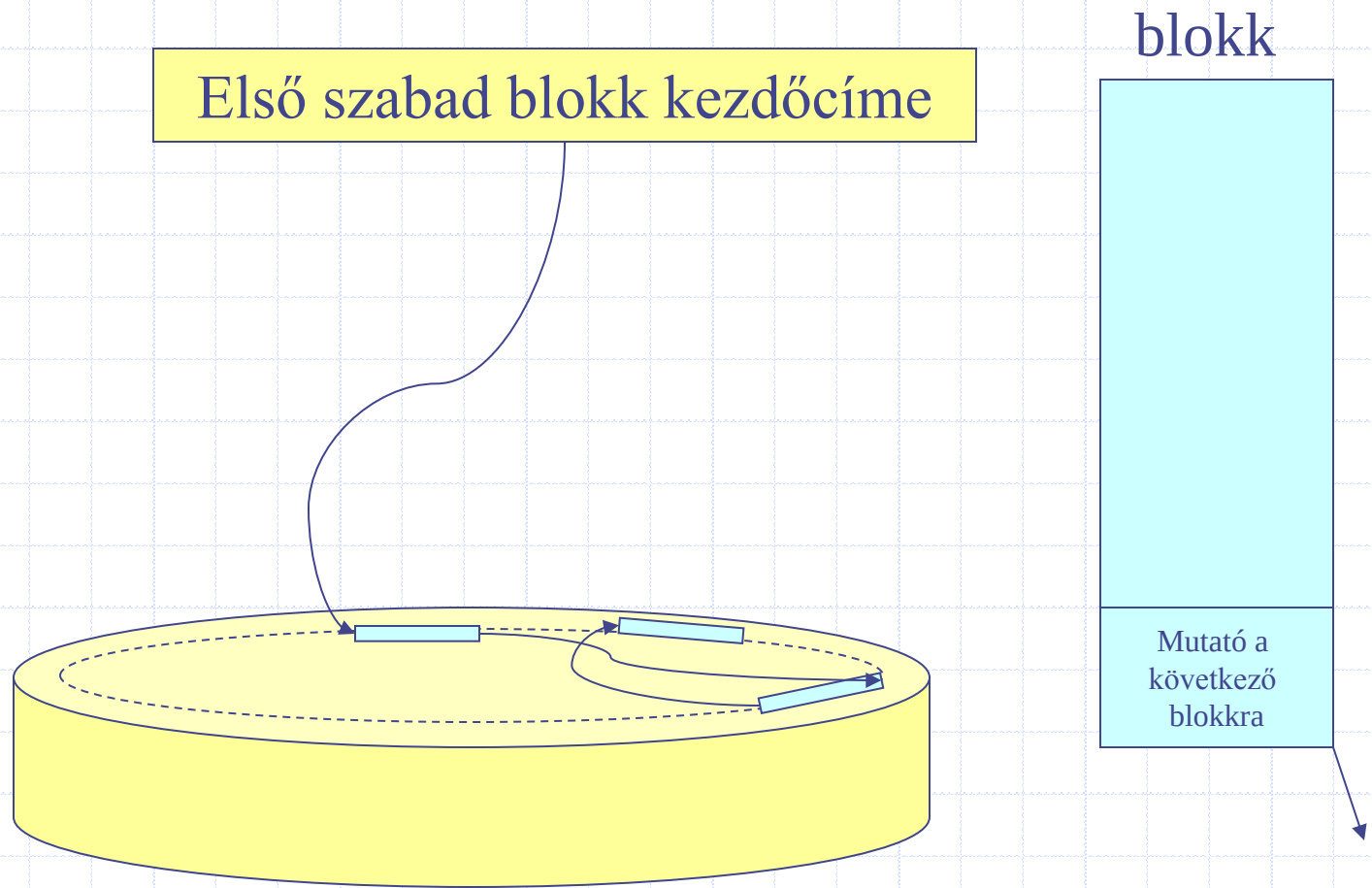
Bittérkép



Láncolt lista

- Az első szabad blokk címének tárolása, arra felfűzve a többi.
- A blokk területéből vesszük le a cím területét.
- Nem hatékony, lassú (sok lemezművelet).

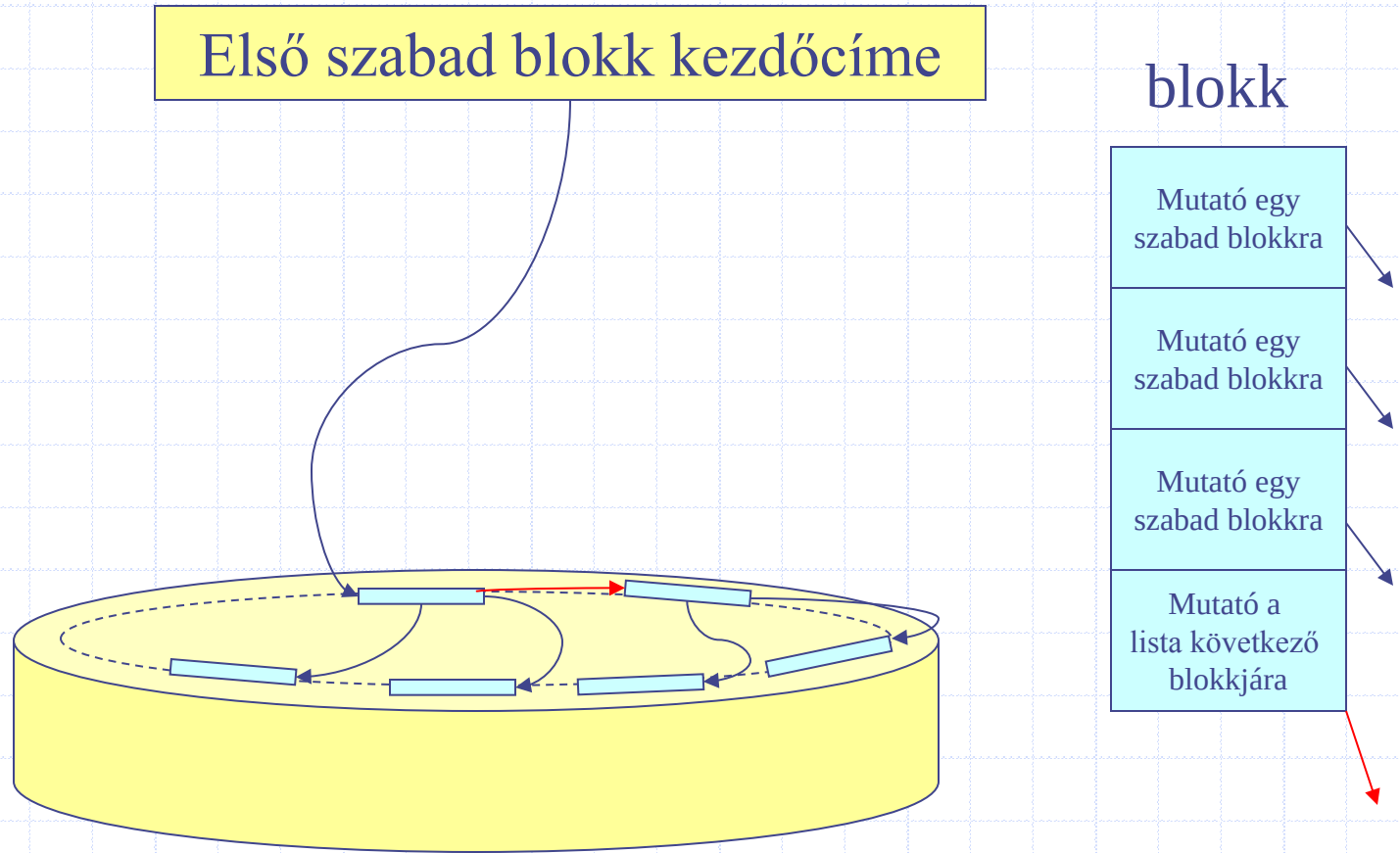
Láncolt lista



Szabad helyek csoportjainak listája

- Láncolt lista javítása.
- Minden blokk n db (ennyi cím fér el a blokkban) szabad blokkra hivatkozik
- $n - 1$ ténylegesen szabad, az n . a lista új elemére mutat.

Szabad helyek csoportjainak listája



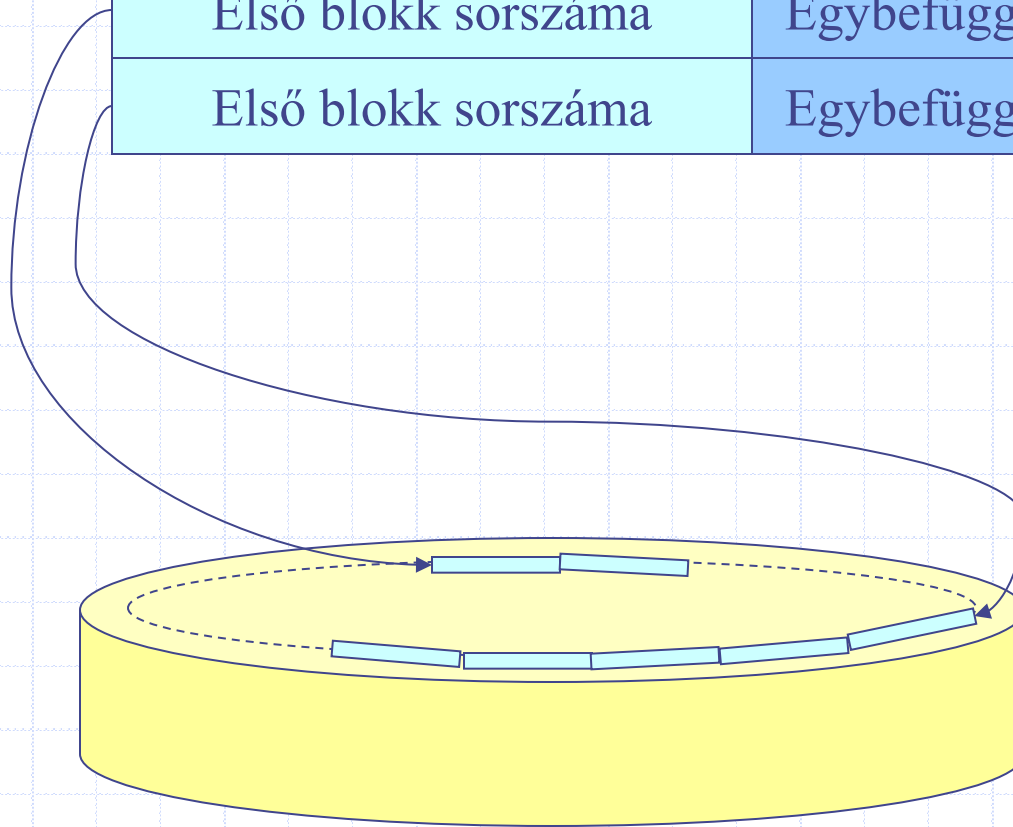
Egybefüggő szabad terület tárolása

- Egy táblázatban tároljuk az összefüggő szabad blokkokat (első blokk sorszáma, blokkok száma)
- Akkor jó, ha az átlagos hossz sokkal nagyobb egynél + egymás mellett akarunk allokálni
- A hosszt lehet kódoltan tárolni (így kisebb helyet foglal) → Huffman kód (~ telefax kódolás): változó hosszúságú, optimális

Egybefüggő szabad terület tárolása

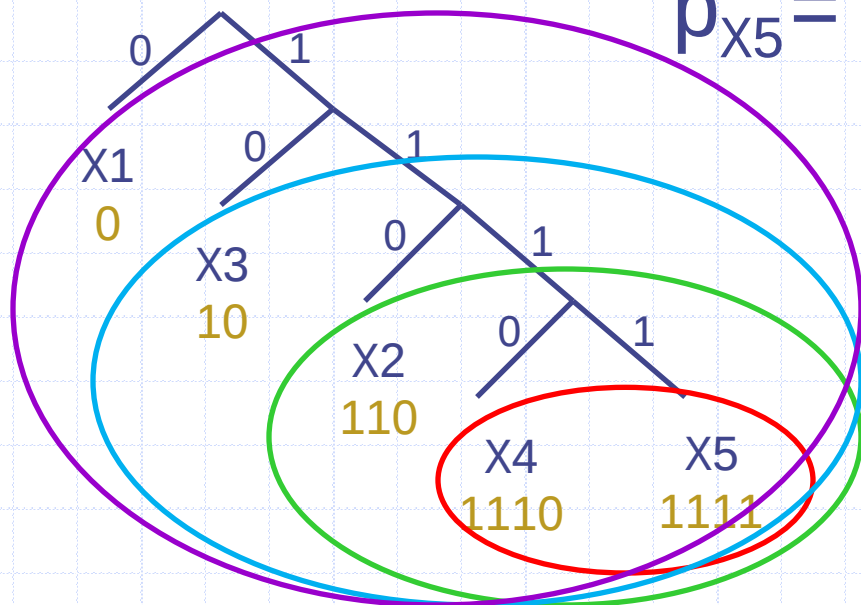
Táblázat

Első blokk sorszáma	Egybefüggő blokkok száma (2)
Első blokk sorszáma	Egybefüggő blokkok száma (5)



Egybefüggő szabad terület tárolása (folyt.) – Huffman kód

- A kódszó hossza az előfordulási valószínűséggel fordítottan arányos
- Cél: „kiegyensúlyozott” fa létrehozása
- Pl. $p_{x_1}=0,5$; $p_{x_2}=0,1$; $p_{x_3}=0,3$; $p_{x_4}=0,03$; $p_{x_5}=0,07$



A legkisebb valószínűségű események a fa aljára → összevonom → így melyik a legkisebb valószínűség, stb.

$p_{x_4}, p_{x_5} \rightarrow$
 $p_{x_4x_5}=0,1, p_{x_2} \rightarrow$
 $p_{x_4x_5x_2}=0,2, p_{x_3} \rightarrow$
 $p_{x_4x_5x_2x_3}=0,5, p_{x_1}$

A lemez blokkjainak allokációja

- Folytonos terület allokációja
- Láncolt tárolás
- Indexelt tárolás
- Kombinált módszerek

Folytonos terület allokációja

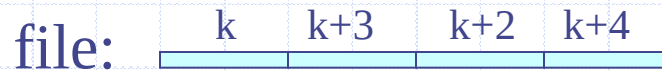
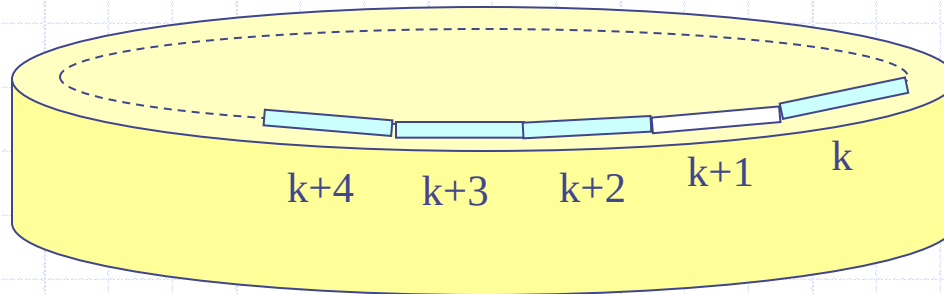
- Az összetartozó információkat egymás melletti blokkokban tároljuk. Első blokk sorszámát és a blokkok számát kell tárolni.
- Hátrányai:
 - Külső tördelődés itt is fennáll
A megoldáshoz itt is használhatók a már megismert algoritmusok (első illeszkedő, legjobban illeszkedő, legrosszabbul illeszkedő).
Nagyméretű tördelődés esetén tömöríteni kell.
 - Sokszor nem tudjuk előre, hogy hány blokkra lesz szükségünk. Lefoglaláskor becsülni kell, a rossz becslés gondot okozhat (hiba és leállás, átmásolni az eddig lefoglalt területet egy másik helyre)
- Előnyei:
 - A tárolt információ soros és közvetlen elérése is lehetséges.
 - Jól használható tárcsere által kirakott szegmensekre (tudjuk előre a méretet)

Láncolt tárolás

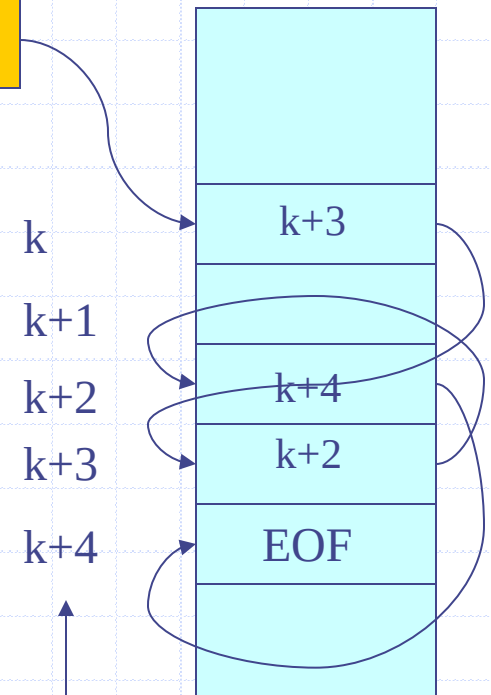
- Blokkokat egyenként allokáljuk, minden blokkban fenntartva egy helyet a következő blokk sorszáma számára (a rendszer az első és az utolsó blokkot tárolja).
- Előnyei:
 - Nincs külső tördelődés
 - Rugalmas, a lefoglalt terület növekedhet.
- Hátrányai:
 - Csak soros elérés lehet.
 - A blokkok sorszámaival nő az állomány mérete. A blokkos másoláskor a sorszámokkal külön kell foglalkozni.
 - Sérülékeny, egyetlen láncszem hibája a tárolt információnak jelentős részének elvesztését jelenti.
- Módosított változata állomány allokációs tábla (file allocation table, FAT)
 - A láncelemeket az állományoktól elkülönítve tároljuk.
 - A szabad helyek tárolására is alkalmas a FAT.

FAT

Könyvtári bejegyzés



FAT



Blokkok sorszámai

Indexelt tárolás

- Az állományhoz tartozó blokkok címei egy indextáblában vannak.
- Előnyei:
 - Közvetlen hozzáférés
 - "Lyukas" állományok tárolása (nem minden blokk tartalmaz valós információt)
- Hátrányai:
 - Az indextábla tárolása legalább egy blokkot elfoglal (kis állományok esetében pazarló)
 - Az indextábla mérete nem ismert, lehetővé kell tenni, hogy az növekedhessen.
 - láncolt indexblokkok
 - többszintű indextábla
 - kombinált mód, kis állományok esetében egy, majd többszintű indextábla

Indexelt tárolás

Könyvtári bejegyzés

név	attribútumok	...	indexblokk
-----	--------------	-----	------------

indexblokk

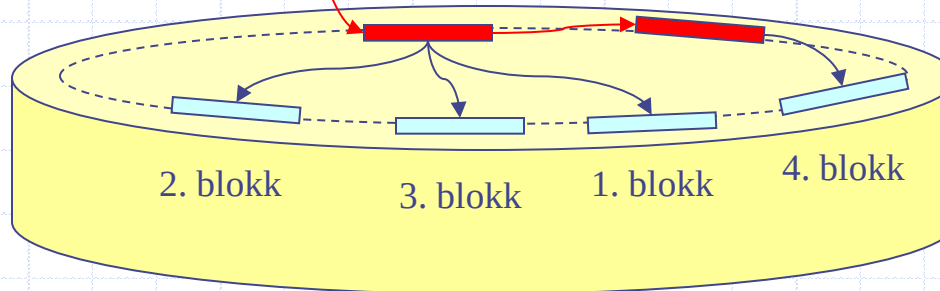
1. blokk száma

2. blokk száma

3. blokk száma

N. blokk száma

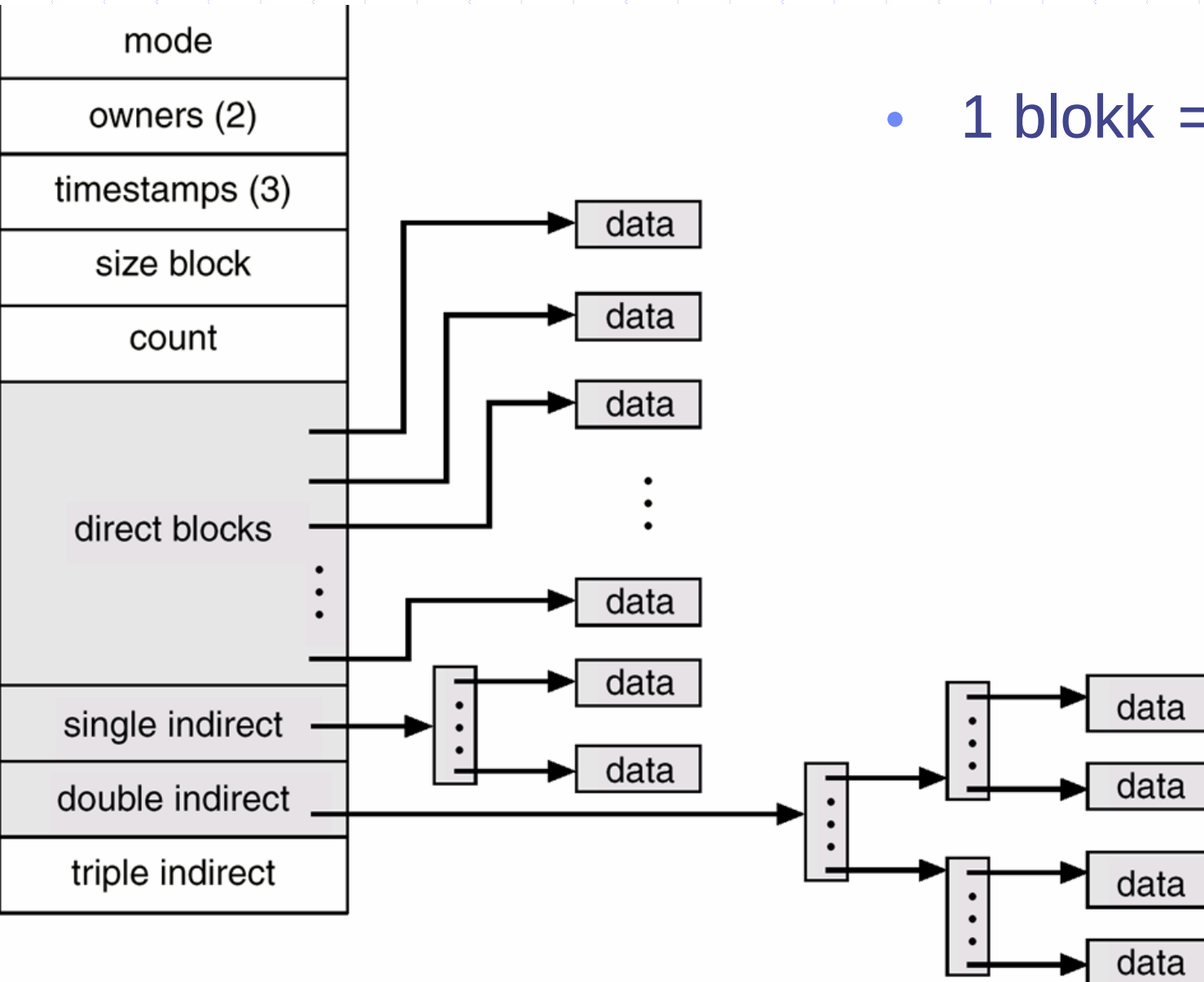
Következő
indexblokk száma



Kombinált módszerek

- hozzáférési módja szerint:
 - Soros hozzáférés esetén láncolt, közvetlen hozzáférés esetén indexelt
- méret szerint:
 - Kis állomány esetén folytonos, nagyok indexelve

Példa: UNIX



- 1 blokk = 4kByte

Az állományok belső szerkezete

- Az állomány egy bitsorozat, amit a felhasználó a saját szempontjai szerint egységekbe csoportosíthat.
 - Mező (field)
több bit, valamilyen típusú adatot ír le (byte-os szervezés általában)
 - Rekord (record)
Mezők csoportja. Egy állomány azonos szerkezetű rekordok gyűjteménye vagy különböző, de azonosítható típusú rekordokból áll.
- Mező és rekord lehet változó hosszúságú (a hossz egyértelműen meghatározható, végjel vagy hossz tárolása).

Az OS viszonya a belső szerkezethez

- Nem foglalkozik vele

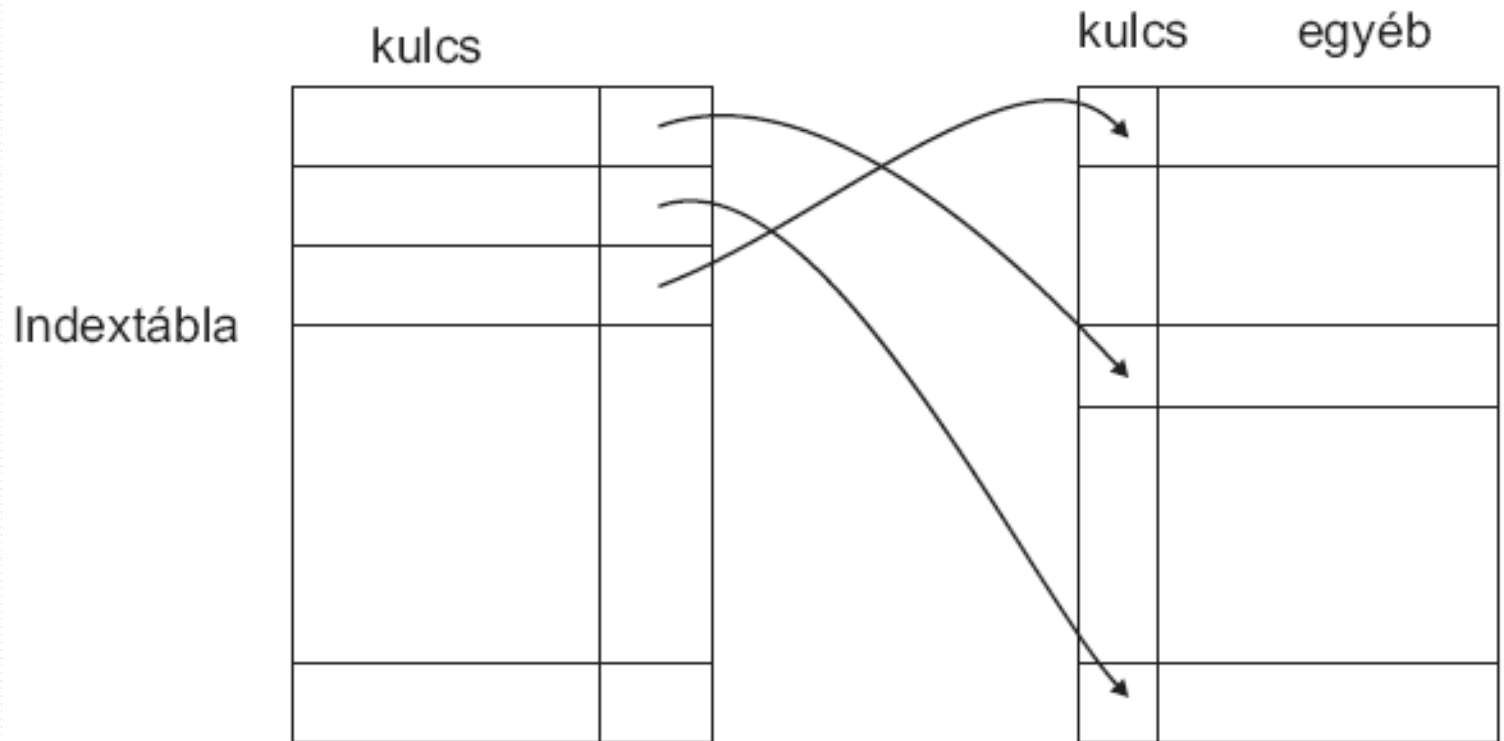
Csak az állományt kezelő programok ismerik az állomány szerkezetét. Egyszerű és rugalmas megoldás.

- Az OS eljárásokkal támogatja
Mezőnkénti vagy rekordonkénti hozzáférési lehetőség az OS által. Nehéz megoldani, bonyolult, általában lehetetlen.

Hozzáférési módok

- **Soros (sequential)**
 - A tárolt információt csak a byte-ok sorrendjében lehet olvasni. Sok feldolgozási feladathoz elegendő a soros hozzáférés.
- **Közvetlen (direct)**
 - A tárolt elemek bármelyikét el lehet érni, ehhez meg kell adni az információ elem állományon belüli sorszámát.
- **Indexelt, index-szekvenciális (index sequential access method, ISAM)**
 - Tartalom szerint akarunk hozzáférni.
 - Kulcs szerint rendezzük az állomány rekordjait. A kulcsok egy index fájlban vannak rendezetten tárolva.
- **Partitionált**
 - Az állományt soros rész-állományok alkotják, az állomány tartalmaz egy nyilvántartást arról, hogy a partíciók hol helyezkednek el a fájlban.

Indexelt, index-szekvenciális



Pl.: árnyilvántartás

Könyvtárak

- Állományok, vagy más könyvtárak gyűjteménye.
- Tartalmát egy nyilvántartás írja le.

Nyilvántartási bejegyzések (directory entry)

A könyvtárban tárolt állományok leírása.

- Az állomány neve:
Könyvtáranként egyedi, homogén (lehet bármilyen karaktersorozat),
részekre bontott (pl, név, típus - extension, verziószám).
- Az állomány fizikai elhelyezkedését leíró információk:
hossz, a hozzá tartozó háttértár blokkjainak leírása, hozzáférés
módja.
- Az állomány kezeléséhez kapcsolódó információk
 - típusa
 - tulajdonosának, létrehozójának (owner) azonosítója
 - időpontok (létrehozás, utolsó módosítás, utolsó hozzáférés,
argumentumának utolsó módosítása, érvényessége)
 - hozzáférési jogosultságok
 - hivatkozás számláló, amennyiben az állományra több különböző néven
és/vagy helyen hivatkoznak

Nyilvántartási bejegyzések (folyt.)

Egyes OS-ek a tárolt információkat kettéválasztják
(UNIX)

- Kötet nyilvántartás (volume directory)
A köteten lévő összes állományt leíró fizikai információk.
- Az állomány nyilvántartások (file directory)
Tartalmazza az állományok könyvtárba szervezésének leírását, valamint az állományokat leíró logikai információkat.

Nyilvántartási bejegyzések (folyt.)

Egyéb információk (az OS központi tárban tárolja):

- az átvitel állapota (folyamatonként)
- soros hozzáférés pozíciója
- megengedett műveletek (a folyamat az állományon milyen műveleteket végezhet)
- osztott kezeléssel kapcsolatos információk
 - hány folyamat használja egyidejűleg
 - kölcsönös kizárás milyen módon kell biztosítani
 - várakozó folyamatok listája

A könyvtárak hierarchiája

- Kétszintű könyvtárszerkezet
 - Egyes felhasználóknak saját könyvtáruk van.
- Faszerkezet
 - Könyvtár tartalmazhat könyvtárat is.
 - Fogalmak:
 - aktuális könyvtár (folyamatonként),
 - gyökér könyvtár,
 - elérési út,
 - keresési utak.

A könyvtárak hierarchiája (folyt.)

- Körmentes irányított gráf
 - Lehetnek linkek is. Linkek mentén viszont nem lehet kör.
 - Link lehet:
 - fizikai: az állomány leíró információkat megismételjük, pl UNIX-nál ugyanarra a kötet-nyilvántartás bejegyzésre hivatkozunk
 - logikai
 - Problémák:
 - egy állományhoz többféle elérés tartozik (mentésnél, statisztikáknál, osztott elérésnél csak egy útvonalat kell figyelembe venni)
 - annak meghatározása, hogy mikor kell egy állományt törölni (egy állományra hivatkozások számát nehéz meghatározni)
- Általános gráf
 - Kör lehetséges
 - Keresésnél végtelen ciklus.
 - Egy törléssel a teljes hurok a "levegőben lógva" maradhat (nem törölt könyvtárak, amik a gyökérből kiindulva nem érhetők el).

Műveletek állományokon

- átvitel, írás, olvasás
 - Közvetlen átvitel esetén, információ címe szükséges
 - Soros hozzáférésnél az aktuális pozíciót a rendszer növeli és tárolja.
 - Szimultán írás-olvasás esetén soros hozzáférésnél közös vagy több pozíció használata.
- hozzáadás (append)
 - Az állomány végéhez új információt írunk. Az állomány mérete növekszik, esetlegesen új blokk lefoglalása.
- pozicionálás
 - Soros hozzáférés esetén megadhatjuk az aktuális pozíciót.

Műveletek állományokon (folyt.)

- állomány megnyitása
 - az állomány megkeresése
 - jogosultságok ellenőrzése
 - az elvégezendő műveletek megadása
 - osztott állománykezelés szabályozása
 - soros hozzáférés pozíciójának beállítása
- A műveletek hatékonyabbak, ha nem kell mindegyiknél az állományt a nyilvántartásból kikeresni.
- A sikeres megnyitás után OS felépít egy adatszerkezetet, a további műveletek erre az adatszerkezetre hivatkoznak.

Műveletek állományokon (folyt.)

- állomány lezárása
 - puffer esetén a ki nem írt információ kiírása, osztott állománykezelésnél az állomány felszabadítása.
- állomány végrehajtása
 - Az OS létrehoz egy új folyamatot, a programállományt betölti a folyamat tárterületére és elindítja.
- A könyvtárra is hatással levő műveletek:
 - állomány létrehozása (új bejegyzés, blokkok lefoglalása)
 - állomány törlése (bejegyzés megszüntetése, blokkok felszabadítása)

Műveletek könyvtárakon

- Állomány attribútumának módosítása (az állományhoz tartozó logikai információk megváltoztatása)
- Új könyvtár létrehozása
- Könyvtár törlése
 - csak akkor törölhető, ha üres
 - törli a benne lévő állományokat
 - rekurzívan törli a benne lévő könyvtárakat is

Műveletek könyvtárakon (folyt.)

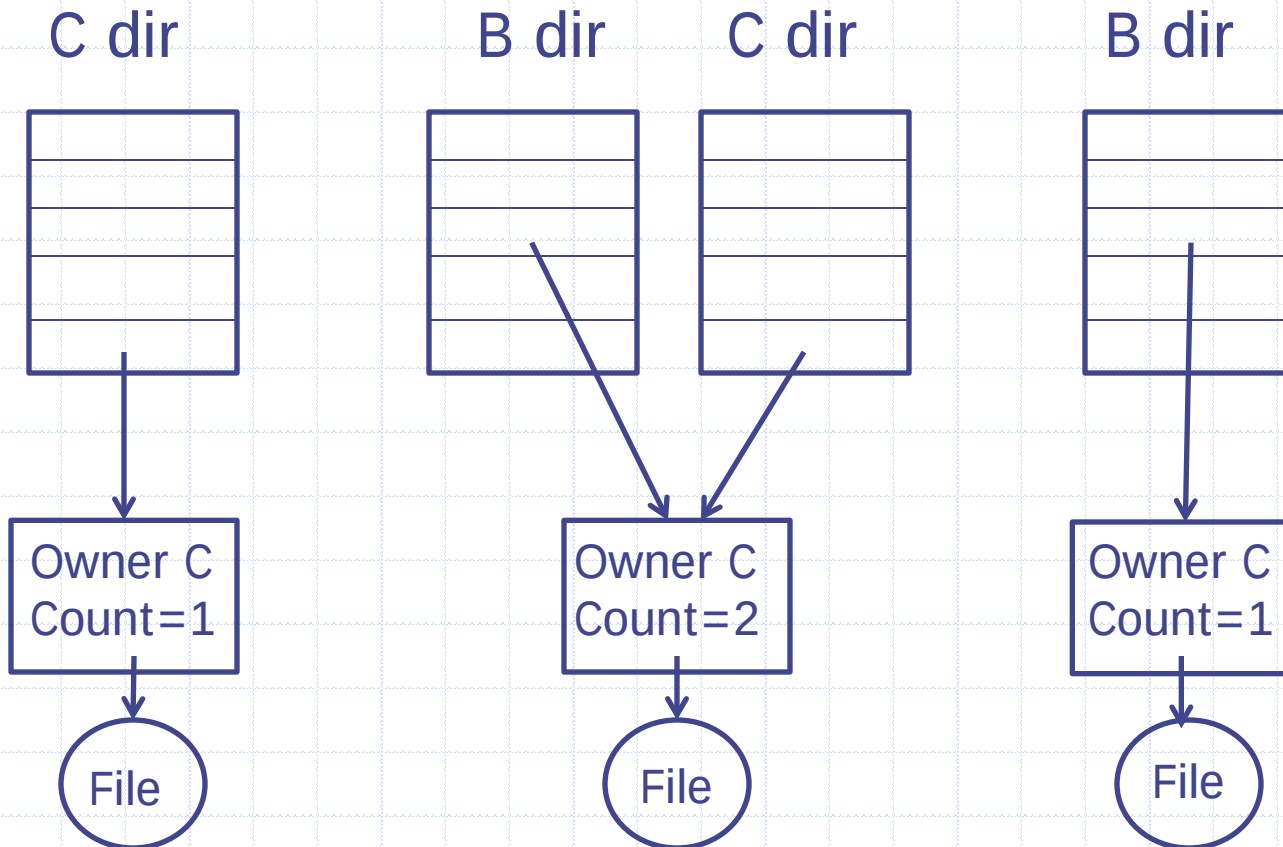
- Keresés
 - Egy névhez meg kell találni a hozzá tartozó állományt. A keresés típusa függ a nyilvántartás szerkezetétől: rendezetlen keresés, rendezett felező keresés, hash keresés.
- Új bejegyzés létrehozása
 - Nyilvántartás méretének dinamikusan növekednie kell. Rendezett nyilvántartás esetén a rendezettséget meg kell tartani.
- Bejegyzés törlése

Osztott állománykezelés

- Ez is egy erőforrás, amelyet egyidejűleg több folyamat is használni akar.
- Csak olvasás esetén osztottan gond nélkül használható.
- Írás esetén kölcsönös kizárással kell védeni a folyamatot.
 - Egész állományra vonatkozó szabályozás (file lock)
 - Az állományt először megnyitó folyamat definiálja, hogy a későbbi megnyitási kérelmekből mit engedélyezhetünk.
 - kizárólagos használat
 - többi folyamat csak olvashatja
 - több folyamat is megnyithatja írási joggal
 - Írás esetén az olvasó folyamatok mikor veszik észre a változást:
 - azonnal,
 - ha a folyamat lezárta az állományt.
 - Állomány részeire vonatkozó kizárás
 - Rekordonként lehet kizárásokat definiálni.

Osztott állománykezelés

- Szimbólikus linkelés: Egy link típusú file-t hoz létre a másik könyvtárban, ami az utat mutatja a file-hoz
- UNIX:
- C dir



A hozzáférés szabályozása

- Cél: jogosulatlanok ne férjenek hozzá állományokhoz, ne végezhessek rajtuk műveleteket. A jogokat a létrehozója, vagy speciális jogokkal rendelkező felhasználó definiálja.
- Jogosultságok:
 - állományokra: írás, olvasás, végrehajtás, hozzáírás, törlés
 - könyvtárakra: könyvtár módosítása, listázás, keresés, új állomány létrehozása, könyvtár törlése.
- Jogosultság tartozhat:
 - állományokhoz
 - elérési útvonalhoz (különböző elérésekhez más-más jogosultságok)
- Jogosultság definiálható:
 - felhasználónként
 - felhasználó csoportonként